Applied Machine Learning

Linear Classifiers

UNIVERSITY OF
GOTHENBURG

CHALMERS

Richard Johansson

richard.johansson@cse.gu.se

preliminaries: binary classification tasks

- we say that a classification task is **binary** if it has two classses
- example:
  - spam / not spam
  - patient has COVID-19 / is healthy
  - equipment is broken / equipment is working properly
  - positive review / negative review

linear classifiers

- a binary **linear classifier** is defined in terms of a scoring function like this

$$\text{score} = \boldsymbol{w} \cdot \boldsymbol{x}$$

- explanation of the parts:
  - $\boldsymbol{x}$ is a vector with features of what we want to classify (e.g. made with a `DictVectorizer`)
  - $\boldsymbol{w}$ is a vector representing which features the classifier thinks are important
  - $\cdot$ is the dot product between the two vectors
- works for **binary** classification tasks
  - return the first class if the score > 0
  - . . . otherwise the second class
  - we will later generalize this to more than 2 classes
- the essential idea: **features are scored independently**

quick note: the bias term

- sometimes, linear classifiers are expressed as

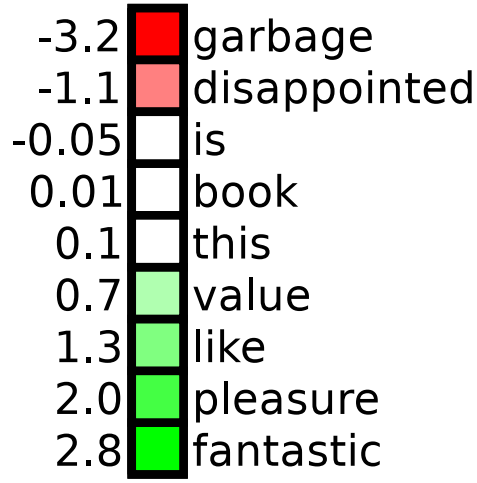
$$\text{score} = \boldsymbol{w} \cdot \boldsymbol{x} + b$$

  where b is called the **offset**, **bias term**, or **intercept**

- for now, we'll ignore b by assuming that $\boldsymbol{x}$ includes a feature that is constant (e.g. always 1)

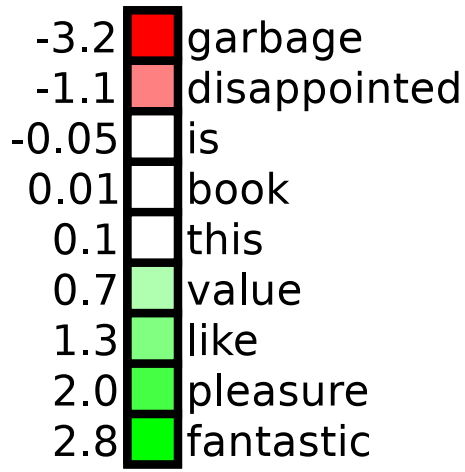
interpreting linear classifiers: example

- each weight in $\boldsymbol{w}$ corresponds to a feature
  - e.g. `"fine"` probably has a high positive weight in sentiment analysis
  - `"boring"` a negative weight
  - `"and"` near zero

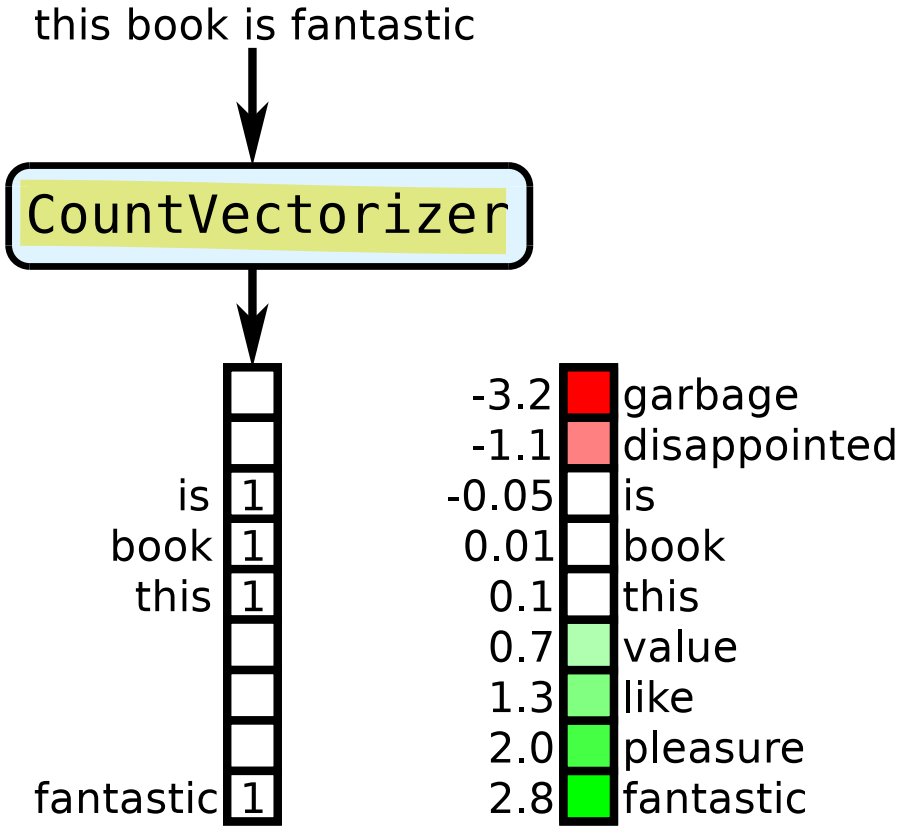


example, continued

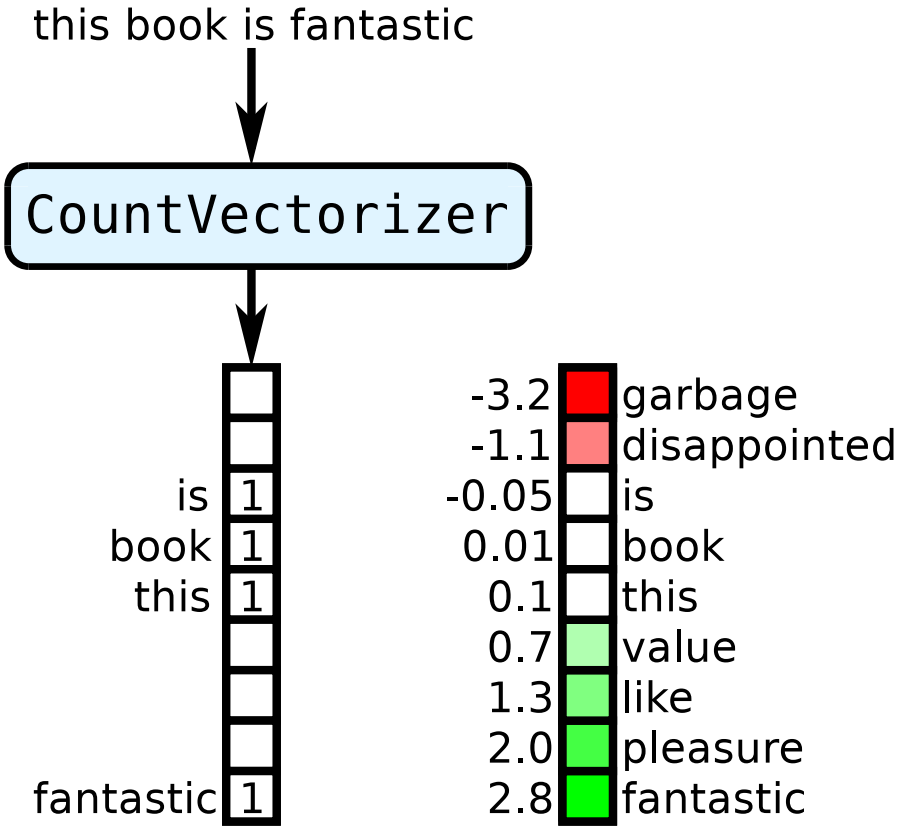
this book is fantastic



example, continued



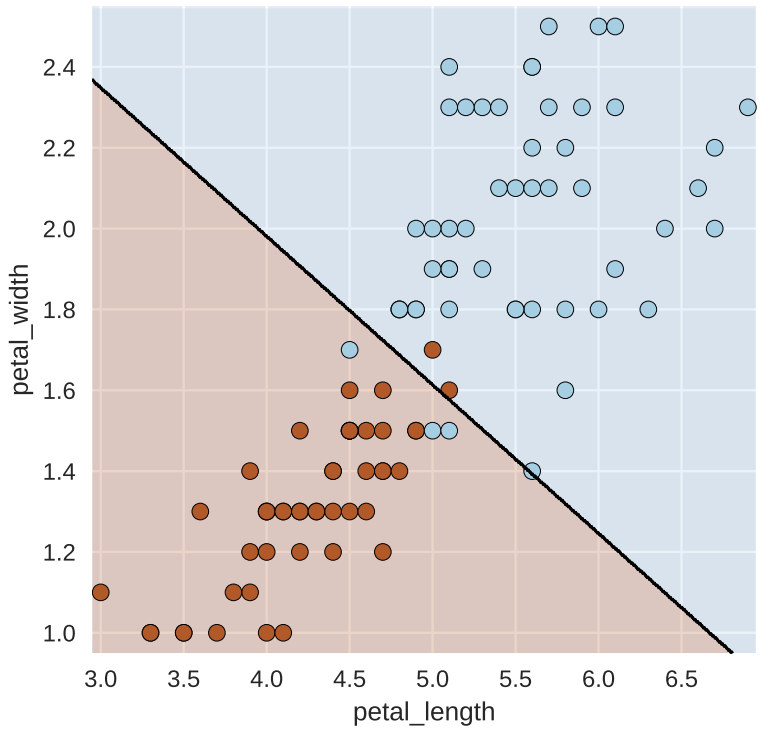
example, continued



score = 1·-0.05 + 1·0.01 + 1·0.1 + 1·2.8 = 2.86

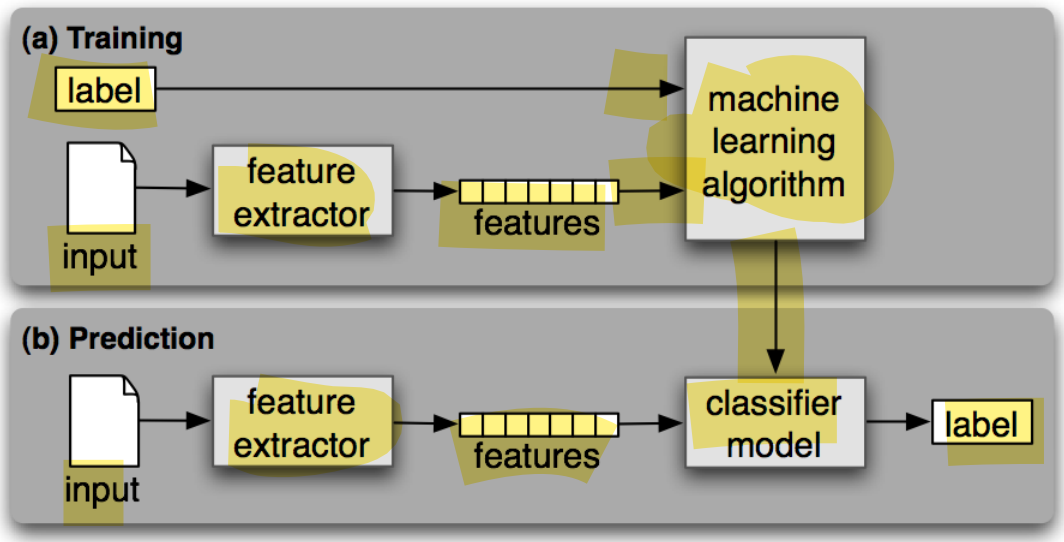
geometric view

- geometrically, a linear classifier can be interpreted as separating the vector space into two regions with a line (plane, hyperplane)



training linear classifiers

- the family of learning algorithms that create linear classifiers is quite large
  - perceptron, Naive Bayes, support vector machine, logistic regression/MaxEnt, ...
- their underlying theoretical motivations can differ a lot but in the end they all return a weight vector $\boldsymbol{w}$



the perceptron learning algorithm

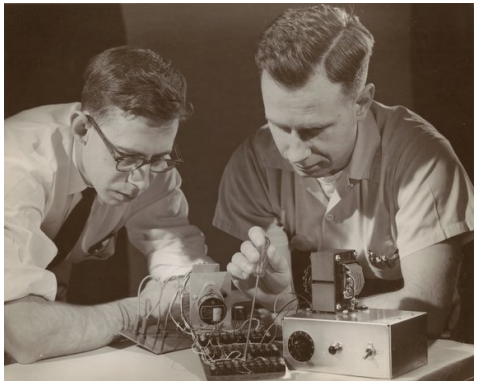
- start with an empty weight vector (all zeros)
- repeat: classify according to the current weight vector
- each time we misclassify, change the weights a bit
  - if a positive instance was misclassified, add its features to the weight vector
  - if a negative instance was misclassified, subtract ...

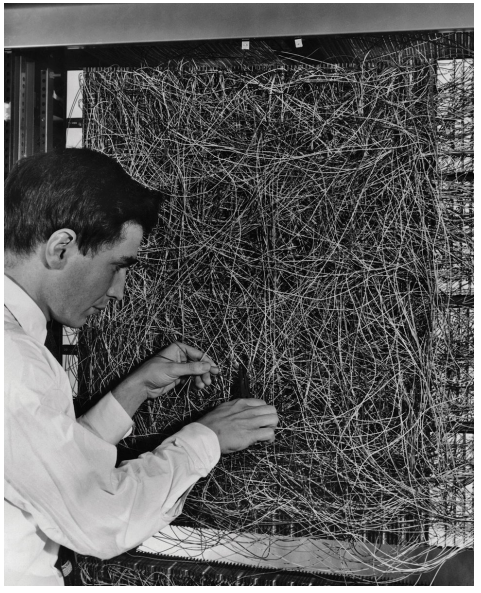
the perceptron learning algorithm, formally

$\boldsymbol{w} = (0, \ldots, 0)$ (← a weight vector of all zeros)
repeat N times
  for $(\boldsymbol{x}_i, y_i)$ in the training set
    score $= \boldsymbol{w} \cdot \boldsymbol{x}_i$
    if a positive instance is misclassified
      $\boldsymbol{w} = \boldsymbol{w} + \boldsymbol{x}_i$
    else if a negative instance is misclassified
      $\boldsymbol{w} = \boldsymbol{w} - \boldsymbol{x}_i$
return $\boldsymbol{w}$

a historical note

- the perceptron was invented in 1957 by Frank Rosenblatt
- initially, a lot of hype!
- the realization of its limitations led to a backlash against machine learning in general
  - the nail in the coffin was the publication in 1969 of the book *Perceptrons* by Minsky and Papert
- new hype in the 1980s, and now...

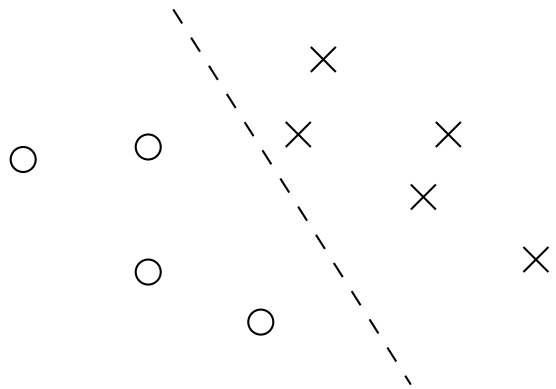


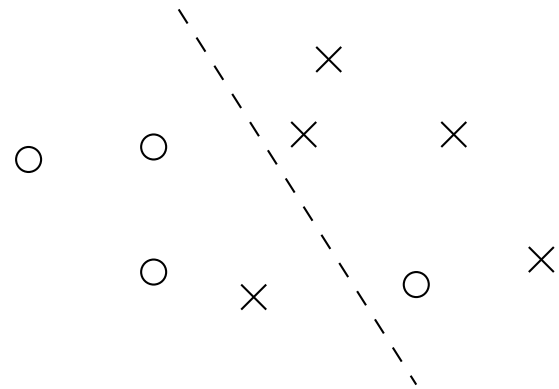


[source]

linear separability

- a dataset is **linearly separable** if there exists a $\boldsymbol{w}$ that gives us perfect classification



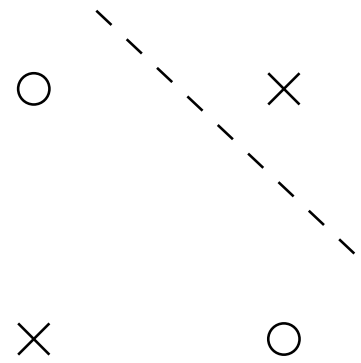


- **theorem**: if the dataset is linearly separable, then the perceptron learning algorithm will find a separating $\boldsymbol{w}$ in a finite number of steps
- what if the dataset is not linearly separable?

a simple example of linear inseparability

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

very good	Positive
very bad	Negative
not good	Negative
not bad	Positive



coding a linear classifier using NumPy

```
class LinearClassifier(object):

    def predict(self, x):
        score = x.dot(self.w)
        if score >= 0.0:
            return self.positive_class
        else:
            return self.negative_class
```

better: handle all instances at the same time

```
class LinearClassifier(object):

    def predict(self, X):
        scores = X.dot(self.w)
        out = numpy.select([scores>=0.0, scores<0.0],
                           [self.positive_class,
                            self.negative_class])
        return out
```

side note: what happened in that code?

```
>>> import numpy

>>> scores = numpy.array([-1, 2, 3, -4, 5])

>>> scores >= 0
array([False,  True,  True, False,  True], dtype=bool)

>>> scores < 0
array([ True, False, False,  True, False], dtype=bool)

>>> numpy.select([scores >= 0, scores < 0], ["positive", "negative"])
array(['negative', 'positive', 'positive', 'negative', 'positive'],
      dtype='|S8')
```

https://docs.scipy.org/doc/numpy-1.15.4/reference/generated/numpy.select.html

perceptron reimplementation in NumPy

```
class Perceptron(LinearClassifier):
    def __init__(self, n_iter=10):
        self.n_iter = n_iter

    def fit(self, X, Y):

        n_features = X.shape[1]
        self.w = numpy.zeros( n_features )

        for i in range(self.n_iter):
            for x, y in zip(X, Y):

                score = self.w.dot(x)

                if score <= 0 and y == self.positive_class:
                    self.w += x
                elif score >= 0 and y == self.negative_class:
                    self.w -= x
```

linear classifiers in scikit-learn

- small selection of learning algorithms:
  - `sklearn.linear_model.Perceptron`
  - `sklearn.linear_model.LogisticRegression` (later)
  - `sklearn.svm.LinearSVC` (later)
- to compute the score function: `model.decision_function(x)`
- after training, weights are stored in the attribute `model.coef_`

take-home messages

- a (binary) **linear classifier** is expressed as

$$\text{score} = \boldsymbol{w} \cdot \boldsymbol{x}$$

  and we return one class if this score is positive, another if it is negative
- interpretation: w_i shows how feature i pulls the decision towards the positive or negative class
- there are many different methods to **train** $\boldsymbol{w}$
- we've now seen the **perceptron** algorithm that operates in a mistake-driven fashion
- if a dataset is **linearly inseparable**, a linear classifier can't deal with it perfectly